On The Porch

Greenfield System Design
Dorchester Community Chatbot — Production Architecture

System Design Document

June 2026

Abstract

This document specifies a production-grade architecture for **On The Porch**, a Dorchester-localized community chatbot intended to serve up to **130,000 residents**. The design optimizes for stability, data freshness, high-quality grounded answers, and **sub-5-second** response latency (p95), while supporting chat, an events panel, guest and registered authentication, and an admin dashboard. The architecture is specified from first principles, independent of any existing implementation.

Contents

1 Product Goals and Constraints **3**
1.1 What We Are Building . . . 3
1.2 Non-Functional Requirements . . . 3
1.3 Key Design Insight . . . 3

2 High-Level Architecture **3**
2.1 Component Overview . . . 3
2.2 Data Flow (Simplified) . . . 3
2.3 Architectural Principles . . . 4

3 Logical Components **4**
3.1 Frontend (Single-Page Application) . . . 4
3.2 API Layer . . . 4
3.3 Chat Orchestrator . . . 5
3.3.1 Query Pipeline (Target: $\leq$ 5s p95) . . . 5
3.3.2 Tier 0 — No LLM ($<$ 100ms) . . . 5
3.3.3 Tier 1 — Intent Classification ($\leq$ 200ms) . . . 5
3.3.4 Tier 2 — Data Fetch (parallel, $\leq$ 800ms) . . . 5
3.3.5 Tier 3 — Answer Synthesis (one LLM call, $\leq$ 2.5s) . . . 6

4 Data Architecture **6**
4.1 Storage Choices . . . 6
4.2 Core Schemas (Simplified) . . . 6
4.3 Ingestion Pipeline . . . 6
4.3.1 Ingestion Rules . . . 6
4.3.2 Ingestion Invariants . . . 6

5 Latency Budget **7**
5.1 Latency Anti-Patterns to Eliminate . . . 7
5.2 Caching Strategy . . . 7

6 Feature-Specific Design **7**
6.1 Chat Authentication . . . 7
6.2 Events Panel . . . 8
6.3 Admin Dashboard . . . 8
6.4 Boston.gov Enrichment (Off Hot Path) . . . 8
6.5 Flagging and Feedback . . . 8

7 Retrieval and Answer Quality **8**
7.1 Hybrid Search Pipeline . . . 8
7.2 Chunking Strategy . . . 9
7.3 Grounding Contract . . . 9
7.4 Safety (911 / Crime) . . . 9

8 Infrastructure and Deployment **9**
8.1 Production Topology . . . 9
8.2 Scaling Estimates (130k Residents) . . . 9
8.3 Environments . . . 10
8.4 CI/CD . . . 10

9 Observability and SLOs **10**
9.1 Golden Signals . . . 10
9.2 Request Tracing . . . 10
9.3 Synthetic Monitoring . . . 10

10 Security Model **10**

11 Phased Build Plan **11**
11.1 Phase 1 — Foundation (4–6 weeks) . . . 11
11.2 Phase 2 — RAG Core (4–6 weeks) . . . 11
11.3 Phase 3 — Admin and Quality (3–4 weeks) . . . 11
11.4 Phase 4 — Scale and Polish (ongoing) . . . 11

12 Anti-Patterns to Avoid **11**

13 Summary **11**

1 Product Goals and Constraints

1.1 What We Are Building

A web application where Dorchester residents can ask natural-language questions about:

- Neighborhood news (DotNews, local RSS, community notes)
- City services (311 requests, response patterns, categories)
- Public safety (911 / crime summaries — responsibly framed)
- Events (calendar, meetings, festivals)
- Policy and civic context (plans, anti-displacement, budgets where available)
- General Dorchester orientation

1.2 Non-Functional Requirements

Requirement	Target
Response latency (p50)	≤ 3 seconds
Response latency (p95)	≤ 5 seconds
Availability	99.9% (API + chat)
Data freshness	311/911 $\leq$ 1h; news $\leq$ 4h; events $\leq$ 24h
Peak concurrent users	2,000+ without degradation
Answer quality	Grounded, cited, refusal when uncertain
Cost predictability	≤ 2 LLM calls per question (normal path)
Privacy	Minimal PII; guest-by-default; auditable admin actions

Table 1: Non-functional requirements

1.3 Key Design Insight

Most community chatbot implementations are slow because each question triggers **3–6 sequential LLM calls** (route → retrieve gate → SQL generation → summarize → merge → fallback). A greenfield design must **cap LLM round-trips at 1–2** and push everything else to **precomputed data, rules, and caches**.

2 High-Level Architecture

2.1 Component Overview

The system comprises six logical planes:

1. **Clients** — Web SPA (mobile-responsive)
2. **Edge** — CDN for static assets; WAF and edge rate limiting
3. **API layer** — Stateless horizontally scaled services behind a load balancer
4. **Hot-path cache** — Redis cluster (sessions, rate limits, semantic cache)
5. **Data plane** — PostgreSQL (OLTP), time-series tables, vector database, object storage
6. **Ingestion** — Async workers scheduled via cron/queue; never on the chat critical path

2.2 Data Flow (Simplified)

```
Browser (SPA)
    -> CDN (static) + WAF -> Load Balancer
        -> Auth Service
```

```
        -> Chat Orchestrator -> Redis / PostgreSQL / Vector DB / LLM APIs
        -> Events API       -> PostgreSQL (materialized events)
        -> Admin API        -> PostgreSQL (RBAC)

Scheduler -> Ingestion Workers -> PostgreSQL / Object Storage / Vector DB
```

2.3 Architectural Principles

1. **Separate read path from write path** — chat never blocks on ingestion.
2. **Precompute before query time** — aggregates, event lists, and embeddings are ready before a user asks.
3. **One orchestrator, many specialists** — single chat service coordinates SQL, vector search, and LLM; no microservice chatter on the hot path.
4. **Fail soft, degrade gracefully** — stale cache with disclaimer beats a 30s hang or HTTP 500.
5. **Grounding is mandatory** — every factual answer cites rows or document IDs used.

3 Logical Components

3.1 Frontend (Single-Page Application)

Technology: React or vanilla JS with Vite; hosted on CDN (Cloudflare Pages, S3+CloudFront, or Vercel).

Surface	Responsibility
Chat workspace	Threads, streaming tokens, sources, flag flow
Events panel	Pre-fetched events; no LLM on panel load
Auth screens	Guest default; optional signup/login
Admin dashboard	Stats, flags, knowledge CRUD, moderation

Client rules for speed:

- Events panel loads from `GET /events` only — never waits on chat.
- Streaming SSE for answers; 5s client abort and retry UX.
- Instant local replies for chitchat (display immediately; persist asynchronously).
- `sessionStorage` for guest Bearer tokens; HttpOnly cookies for registered users.

3.2 API Layer

Technology: Python FastAPI or Flask behind Gunicorn/Uvicorn; 3–8 replicas with autoscaling on CPU and latency.

Service	Endpoints	Notes
Auth	`/auth/*`	Guest Bearer + CSRF; registered cookie session
Conversations	CRUD threads/messages	Optimistic UI; message length limits
Chat	`POST /chat`, `POST /chat/stream`	Core orchestrator
Events	`GET /events`	Materialized table only
Admin	stats, flags, knowledge	RBAC + CSRF on mutations
Health	`/health`, `/ready`	DB, Redis, vector, ingestion lag

3.3 Chat Orchestrator

The chat orchestrator is the only service on the critical path for answer quality and latency.

3.3.1 Query Pipeline (Target: $\leq$ 5s p95)

```
User question
  -> Tier 0: rules / exact cache / FAQ        [< 100 ms, no LLM]
  -> Tier 1: intent classification            [< 200 ms]
  -> Tier 2: parallel SQL + retrieval         [< 800 ms]
  -> Tier 3: single LLM synthesis (streamed)  [< 2500 ms]
  -> Post-process: citations + safety         [< 200 ms]
```

3.3.2 Tier 0 — No LLM ($<$ 100ms)

- Chitchat, help, identity questions
- Exact + semantic cache (Redis) for repeated questions
- Pre-authored FAQ for top 50 Dorchester questions (admin-maintained)

3.3.3 Tier 1 — Intent Classification ($\leq$ 200ms)

Use a **hybrid classifier**, not a generative LLM router:

Signal	Method
Keywords	311, crime, event, meeting, DotNews, street names
Patterns	Regex for dates, "this week", "last month"
Light model	Small classifier (Gemini Flash) $\rightarrow$ intent label
Confidence	If low $\rightarrow$ `hybrid_light` with tight timeouts

Intent taxonomy (fixed, $\sim$12 labels):

```
events_list | events_detail | 311_stats | 911_stats | safety_trend |
news_recent | policy_doc | neighborhood_fact | city_service_howto |
community_note | chitchat | unknown
```

Critical rule: Classification must not be a separate generative LLM call with JSON-parse fallback to hybrid (the slowest path).

3.3.4 Tier 2 — Data Fetch (parallel, $\leq$ 800ms)

Intent	Data Source	Technique
`events_list`	`events_materialized`	Zero LLM; SQL + template
`311_stats`	Aggregated 311 views	Parameterized SQL only
`911_stats`	Aggregated 911 views	SQL + safety copy rules
`news_recent`	Vector + BM25	Top-k=8, parallel sources
`policy_doc`	Policies collection	Higher k, rerank top 5
`hybrid_light`	SQL slice + vector slice	Parallel execution

Never generate SQL with an LLM on the hot path for common intents. Use a **prepared query library** (20–40 named queries). LLM$\rightarrow$SQL only for novel analytical questions, with 2s timeout and graceful fallback.

3.3.5 Tier 3 — Answer Synthesis (one LLM call, ≤ 2.5s)

- Single main model call with question + structured context JSON + citation instructions
- Fast capable model (e.g. Gemini 2.5 Flash); Pro reserved for admin tools
- Streaming starts as soon as first tokens arrive
- Hard timeout: 3s → partial answer or cached stale response

4 Data Architecture

4.1 Storage Choices

Store	Purpose	Rationale
PostgreSQL 16	Users, sessions, threads, flags, events	ACID, JSONB, mature ops
PostgreSQL / Timescale	311, 911 time-series	Fast range queries, rollups
Qdrant or pgvector	Document embeddings	Horizontal scale; no SQLite locks
Redis Cluster	Sessions, rate limits, caches	Sub-ms; shared across replicas
S3-compatible	Raw PDFs, newsletters	Cheap, versioned
OpenSearch (optional)	BM25 keyword search	Better recall for street names

Avoid: File-based Chroma on SQLite for production at community scale — concurrent ingestion causes lock contention.

4.2 Core Schemas (Simplified)

OLTP (PostgreSQL):

```
users(id, email, username, role, is_guest, ...)
web_sessions(id, user_id, token_hash, expires_at, ...)
conversation_threads(id, user_id, title, state_json, ...)
conversation_messages(id, thread_id, role, content, sources_json, ...)
interaction_log(id, user_id, query, response, flagged, ...)
admin_knowledge(id, content, category, active, expires_at, ...)
```

Civic data (partitioned by month):

```
requests_311(id, case_id, open_dt, neighborhood, lat, lon, subject, status, ...)
incidents_911(id, incident_id, occurred_at, district, offense_type, ...)
```

Materialized views (refreshed by ingestion):

```
mv_311_weekly_by_neighborhood
mv_911_monthly_by_offense
events_materialized   -- next 30 days, denormalized for API
```

Vector collections (Qdrant):

```
news_articles, policies, transcripts, community_notes, boston_gov_cache
```

4.3 Ingestion Pipeline

4.3.1 Ingestion Rules

4.3.2 Ingestion Invariants

- Never `TRUNCATE` production tables during live operation.
- Bulk vector rebuilds use **blue-green indexing**: build `index_v2`, swap alias.
- Ingestion lag exposed in `/health` for operational visibility.

Source	Frequency	Strategy
311 / 911	Every 30–60 min	Incremental upsert by external ID
RSS / news	Every 1–4 hours	ETag / Last-Modified; dedupe by URL hash
Events	Daily + on-demand	Normalize to `events_materialized`
Policies / PDFs	Weekly or webhook	Chunk, embed, upsert
Wayback backfill	Offline batch only	Staging index; not daily cron

5 Latency Budget

Stage	Budget	Technique
Auth + rate limit	50 ms	Redis session cache
Load thread context	80 ms	Last 6 turns; state in `thread.state_json`
Intent classify	150 ms	Rules first; tiny model if needed
Retrieval / SQL	800 ms	Parallel; prepared SQL; connection pool
LLM synthesis	2500 ms	One call; streaming; fast model
Persist + log	100 ms	Async write-behind for non-critical logs
Total p95	**~3.7 s**	Buffer to 5 s target

Table 2: End-to-end latency budget

5.1 Latency Anti-Patterns to Eliminate

Anti-pattern	Replacement
LLM routing + needs-new-data gate + answer	Single classify + one synthesis
LLM-generated SQL per query	Prepared query catalog
Sequential RAG sources	Parallel with 600ms per-source timeout
Boston.gov scrape on hot path	Async enrichment or nightly cache
Per-request schema introspection	Schema cached 24h in Redis
Embedding on every cache miss	Batch embeddings in ingestion only

5.2 Caching Strategy

Never cache: time-sensitive queries (“today”, “tonight”), unresolved pronoun follow-ups.

6 Feature-Specific Design

6.1 Chat Authentication

Guest model:

- `POST /auth/guest` → ephemeral user + Bearer token (tab-scoped)
- Rate limit: 30 guests/hour/IP; 60 messages/hour/session
- No thread migration on login (fresh registered session)

Registered model:

- HttpOnly session cookie + CSRF double-submit
- Threads persist across devices; optional password auth

Conversation state example:

Layer	Key	TTL	Goal
L1 Exact	`hash(normalize(q))`	15 min	10–20% hit rate
L2 Semantic	embedding ≥ 0.92	15 min	5–15% hit rate
L3 Intent+params	`intent:nbhd:week`	1 hour	20% for stats queries

```
{
  "last_intent": "311_stats",
  "last_neighborhood": "Dorchester",
  "last_sql_result_id": "mv_311_weekly:week12",
  "retrieval_doc_ids": ["news_123", "policy_45"]
}
```

6.2 Events Panel

Design rule: Events panel is **100% decoupled** from the LLM.

```
GET /events?days_ahead=14&limit=20&neighborhood=dorchester
  -> reads events_materialized only
  -> p95 < 100 ms
```

Panel prefetches on app load (parallel with `/auth/me`). Client caches 5 minutes with ETag support.

6.3 Admin Dashboard

Module	Backend	Auth
Stats	Read replicas + pre-aggregated counters	`role=admin`
Flagged responses	`interaction_log WHERE flagged`	admin + CSRF
Knowledge base	CRUD → embed worker	admin + CSRF
Community notes	Approve queue → embed	admin + CSRF
Audit log	Immutable `admin_audit_log`	admin

Guests are hard-blocked at API (`403 guest_not_allowed`) and UI bootstrap.

6.4 Boston.gov Enrichment (Off Hot Path)

When synthesis confidence is low:

1. Return initial streamed answer immediately.
2. Trigger async job to fetch Boston.gov / city page.
3. Push `correction` SSE event or "Updated answer" toast.

6.5 Flagging and Feedback

- `PUT /interactions/:id/flag` with ownership verification
- Flags feed admin queue and optional retraining corpus
- Rate-limit flags to prevent abuse

7 Retrieval and Answer Quality

7.1 Hybrid Search Pipeline

1. BM25 top 20 (exact street names, source names)

2. Vector top 20 (semantic)
3. Reciprocal Rank Fusion → top 8
4. Cross-encoder rerank (optional, ≤ 200ms) for `policy_doc` intent
5. Pass top 5 chunks to synthesis LLM

7.2 Chunking Strategy

Content	Chunk Size	Metadata
News	400–600 tokens	`published_at`, `source`, `url`
Policy PDFs	800 tokens, overlap 100	`section`, `page`
Transcripts	Speaker-aware segments	`meeting_date`
Community notes	Whole note if < 300 tokens	`expires_at`, `category`

7.3 Grounding Contract

Every synthesis prompt enforces:

- Use only provided `CONTEXT`; cite sources as [1], [2].
- If `CONTEXT` is insufficient, state what is missing.
- Never invent statistics; never expose raw addresses from 911 rows.

7.4 Safety (911 / Crime)

- Aggregate by neighborhood and time — no voyeuristic incident narratives.
- Suppress counts < 5 in small geographic areas.
- Fixed disclaimer on safety answers; human review queue for flagged responses.

8 Infrastructure and Deployment

8.1 Production Topology

```
Cloudflare CDN + WAF
    -> Load Balancer
        -> API Pods (N replicas, autoscaling)
            -> Redis (managed cluster)
            -> RDS PostgreSQL (+ read replica for admin)
            -> Qdrant Cloud / self-hosted
            -> Object Storage (S3)
        -> Ingestion Workers (2+ replicas, separate deploy)
            -> same data plane
```

8.2 Scaling Estimates (130k Residents)

Metric	Conservative	Peak Planning
MAU (5–10%)	6,500–13,000	—
DAU (1–3%)	1,300–3,900	—
Peak concurrent	200–500	2,000
Chat QPS (peak)	20–50	100–150
API replicas	3–4	8–12 (autoscale)

8.3 Environments

- **local** — Docker Compose: PostgreSQL, Redis, Qdrant, API, static frontend
- **staging** — Full ingest on subset; synthetic load tests
- **prod** — Blue-green deploy; read replica for admin stats

8.4 CI/CD

- PR: lint, unit tests, contract tests, migration dry-run
- Main: build container → deploy API → migrations → smoke `/health` + golden questions
- Ingestion: separate deploy cadence; feature flags for new sources

9 Observability and SLOs

9.1 Golden Signals

Signal	Tool	Alert Threshold
Latency p50/p95	Prometheus + Grafana	p95 > 5s for 5 min
Error rate	Sentry	> 1% HTTP 5xx
LLM timeout rate	Custom metric	> 5%
Cache hit rate	Redis metrics	< 15%
Ingestion lag	Worker heartbeat	311 > 2h stale
Vector index age	Qdrant metadata	news > 24h stale

9.2 Request Tracing

Every chat request receives `X-Request-ID` with structured fields:

```
intent, tier, cache_hit, sql_ms, retrieval_ms, llm_ms, total_ms, token_count
```

9.3 Synthetic Monitoring

Every 5 minutes, a bot asks 10 golden questions (events, 311, news) and asserts status 200, latency < 5s, and presence of citations.

10 Security Model

Area	Design
Secrets	Vault / cloud secret manager; never in repository
Sessions	HMAC-hashed tokens; quarterly pepper rotation
CSRF	All cookie-authenticated mutations
Rate limits	IP + session + global; Redis sliding window
Input limits	2k chars/message; 4k community notes
Admin	RBAC, audit log, no guest access
LLM	Prompt injection defenses; context delimiters

11 Phased Build Plan

11.1 Phase 1 — Foundation (4–6 weeks)

- PostgreSQL schema; guest + password auth; thread CRUD
- `events_materialized` + Events API + panel
- 311/911 ingest + 3 prepared SQL queries
- Rule-based chitchat + `/health`
- **SLO:** events < 100ms; SQL answers < 2s

11.2 Phase 2 — RAG Core (4–6 weeks)

- Qdrant + news/policy ingest + hybrid retrieval
- Single-call synthesis with citations; SSE streaming
- Semantic + exact cache (Redis)

11.3 Phase 3 — Admin and Quality (3–4 weeks)

- Admin dashboard, flags, knowledge base, community notes approval
- Safety filters on 911 answers
- Golden question suite + load test to 100 QPS

11.4 Phase 4 — Scale and Polish (ongoing)

- Blue-green vector index; async Boston.gov enrichment
- Read replica for admin; autoscaling; cost dashboards
- Optional mobile PWA

12 Anti-Patterns to Avoid

Don't	Do Instead
3–6 LLM calls per question	Max 2; often 0–1
LLM-generated SQL on every query	Prepared query catalog
Chroma SQLite on local disk in prod	Qdrant or pgvector
Daily Wayback crawl in live cron	Offline backfill jobs
Boston.gov scrape before returning answer	Async enrichment
In-memory cache per worker	Redis cluster
Guest threads migrate on login	Fresh session on login
Track vector DB in git	S3 + managed index
Flask debug in production	Container + Gunicorn; `DEBUG=false`

13 Summary

The optimal Dorchester chatbot for **130,000 residents** is not a bigger LLM pipeline. It is:

1. **Precomputed civic data** (311, 911, events) served by SQL and templates for common questions.
2. **One fast synthesis call** with parallel retrieval, not a chain of routing models.
3. **Managed infrastructure** (PostgreSQL, Redis, Qdrant) — not file-locked SQLite vector stores.
4. **Decoupled surfaces** — events panel and admin never compete with chat for LLM time.

5. **Aggressive caching** with clear freshness SLAs and visible degradation.
6. **Async ingestion** with incremental upserts and blue-green index swaps.

Built this way, **p50 $\approx$ 1.5–2.5s** and **p95 $\leq$ 5s** is achievable for the majority of resident questions, while keeping the system stable, updatable, and operable at community scale.